# Teleoperación con Sensado de Fuerza Física entre dos xArm Lite 6: Diseño, Implementación y Evaluación Experimental

Josue Ureña Valencia*, César Arellano Arellano*, Jose Eduardo Sanchez Martinez*, Rafael André Gamiz Salazar*
*Computational Robotics Lab, Tecnológico de Monterrey, Monterrey, N.L., México
{A01738940, A00839373, A01738476, A00838280}@tec.mx

***Resumen*—En este trabajo se implementó un sistema de teleoperación bilateral con sensado de fuerza entre dos manipuladores xArm Lite 6. Un operador mueve el robot maestro manualmente, y el esclavo replica su movimiento en tiempo real. Cuando el esclavo entra en contacto con un objeto, un sensor de fuerza construido con un resorte y un sensor de distancia VL53L0X detecta la compresión y bloquea el maestro, dando al operador retroalimentación táctil de la interacción. La comunicación entre nodos se realizó sobre ROS 2, con el sensor publicando vía micro-ROS desde un ESP32. Los experimentos mostraron seguimiento articular funcional y detección clara de eventos de contacto, validando la arquitectura propuesta como una alternativa de bajo costo a sistemas hápticos comerciales.**

***Index Terms*—Teleoperación, sensado de fuerza, xArm Lite 6, control de impedancia, retroalimentación háptica, micro-ROS, MoveIt.**

## I. Introducción

La teleoperación con retroalimentación de fuerza constituye un paradigma fundamental en robótica cuando se requiere que un operador humano interactúe con entornos remotos o peligrosos sin presencia física [6]. El xArm Lite 6, desarrollado por UFACTORY [1], es un manipulador de 6 grados de libertad (DOF) con capacidad de control de torque que lo hace idóneo para experimentos de sensado de fuerza por torque articular.

**(i) Contexto: Motivación**

El reto central de este proyecto es cerrar el lazo de fuerza entre dos brazos robóticos: el robot **Maestro**, operado por el humano, y el robot **Esclavo**, que interactúa con el entorno. La fuerza de contacto estimada en el esclavo debe reflejarse como torque en las articulaciones del maestro, creando la sensación háptica que guía al operador.

### *I-A. Contexto y Motivación*

La teleoperación con sensado de fuerza es una tecnología importante porque permite que un operador controle un sistema robótico remoto y, al mismo tiempo, reciba información sobre la interacción física con el entorno. Esto es importante porque no basta con ver lo que hace el robot; en muchas tareas también es necesario sentir cuándo existe contacto, cuánta fuerza se está aplicando y qué tan rígido o delicado es el objeto con la que se interactúa. En robótica moderna, este tipo de capacidades se relaciona directamente con un mejor control, mayor precisión y una operación más segura, especialmente en tareas donde un error de contacto puede afectar el resultado del proceso [9]. Además, la teleoperación sigue siendo un tema vigente de investigación por su utilidad en sistemas complejos donde el ser humano aporta decisión y percepción, mientras el robot aporta alcance, repetibilidad y trabajo en entornos inaccesibles o riesgosos [11].

Una aplicación médica que utiliza esta tecnología es la cirugía laparoscópica (método para operar el abdomen y la pelvis utilizando pequeñas incisiones) asistida por robot. En este contexto, el sensado de fuerza y la retroalimentación háptica ayudan a mejorar la manipulación de tejidos, reducir el riesgo de aplicar fuerzas excesivas y darle al cirujano una percepción más cercana a la interacción real, algo especialmente valioso en procedimientos mínimamente invasivos [10]. Por otro lado, una aplicación industrial aparece en la teleoperación de robots para tareas de manipulación remota, ensamblaje o intervención en entornos peligrosos, donde el operador necesita controlar al robot con precisión y detectar contacto de manera confiable para evitar daños al sistema o al entorno [11].

### *I-B. Contribuciones del Trabajo*

- Se diseñó e implementó una arquitectura de teleoperación maestro-esclavo basada en ROS2, en la que el movimiento del robot maestro se transmite al robot esclavo mediante velocidades articulares publicadas en tiempo real.
- Se desarrolló un nodo maestro capaz de calcular el Jacobiano geométrico del xArm Lite 6 a partir de sus parámetros DH, permitiendo transformar una fuerza cartesiana externa en torques articulares mediante la relación $\tau = J^\top F$.
- Se integró un sistema de sensado de fuerza externo con un ESP32 y un sensor VL53L0X, donde la fuerza de contacto se estimó a partir de la deformación de un resorte usando la ley de Hooke y posteriormente se publicó en ROS 2 mediante micro-ROS.
- Se implementó una estrategia de retroalimentación háptica simplificada en el robot maestro, en la que la fuerza medida en el entorno remoto se refleja como esfuerzo

articular, con el objetivo de que el operador perciba el contacto generado por el robot esclavo.

- Se construyó una base experimental funcional que integra la obtención de fuerza, comunicación entre nodos y seguimiento articular, la cual sirve como plataforma para evaluar desempeño, estabilidad y calidad de la interacción háptica.

### *I-C. Estructura del Artículo*

El resto de este artículo se organiza como sigue: la Sección II presenta los fundamentos teóricos; la Sección III describe la arquitectura del sistema; la Sección IV detalla la estrategia de sensado de fuerza; la Sección V expone el esquema de control; la Sección VI documenta la implementación con xArm SDK; la Sección VII reporta los resultados experimentales; la Sección VIII discute los hallazgos; y la Sección IX concluye el trabajo.

## II. Marco Teórico

### *II-A. Cinemática y Jacobiano del xArm Lite 6*

El xArm Lite 6 es un manipulador de $n = 6$ DOF con articulaciones rotacionales. Su cinemática directa se describe mediante la convención Denavit-Hartenberg (DH) o Product of Exponentials (PoE).

**(i) Contexto: Jacobiano Geométrico**

El Jacobiano $\boldsymbol{J}(\boldsymbol{q}) \in \mathbb{R}^{6\times 6}$ mapea velocidades articulares a velocidades cartesianas del efector final:

$$\begin{pmatrix} \boldsymbol{v} \\ \boldsymbol{\omega} \end{pmatrix} = \boldsymbol{J}(\boldsymbol{q})\,\dot{\boldsymbol{q}}$$

Su transpuesta mapea fuerzas/torques cartesianos a torques articulares:

$$\boldsymbol{\tau} = \boldsymbol{J}^{\top}(\boldsymbol{q})\,\boldsymbol{F}_{ext}$$

Esta relación es la base del *torque-based force sensing*.

*Pregunta T1* ***[8 pts]****:*

(a) Expliquen la diferencia entre el **Jacobiano analítico** y el **Jacobiano geométrico** para el xArm Lite 6. ¿Cuál utilizan en su implementación y por qué?

(b) Identifiquen y describan **dos configuraciones singulares** del xArm Lite 6 a partir de su geometría DH. ¿Qué ocurre con la estimación de fuerza $\hat{\boldsymbol{F}} = (\boldsymbol{J}^{\top})^{+}\boldsymbol{\tau}$ cerca de esas singularidades?

(c) ¿Cómo detectan programáticamente que el robot está cerca de una singularidad? Escriban la condición en términos del número de condición $\kappa(\boldsymbol{J})$.

- El Jacobiano geométrico relaciona la velocidad articular $\dot{q}$ con la velocidad lineal y angular del efector final, es decir, con el vector de velocidad espacial o twist. En cambio, el Jacobiano analítico relaciona $\dot{q}$ con la derivada de una parametrización explícita de la pose. Por eso, el Jacobiano analítico depende de cómo se describa la orientación y puede presentar singularidades El Jacobiano analítico relaciona $\dot{\boldsymbol{q}}$ con la variación de la orientación expresada mediante una representación específica, como ángulos de Euler. Esto introduce singularidades propias de esa representación — como el gimbal lock — que no corresponden a ninguna limitación física del robot. El Jacobiano geométrico, en cambio, se construye directamente desde los ejes articulares $\hat{z}_i$ y las posiciones de cada junta, relacionando $\dot{\boldsymbol{q}}$ con la velocidad lineal y angular real del efector [2]. En nuestra implementación utilizamos el Jacobiano geométrico, ya que el objetivo es mapear una fuerza cartesiana externa hacia torques articulares mediante $\boldsymbol{\tau} = \boldsymbol{J}^{\top}\boldsymbol{F}$.
  Esta relación no requiere ninguna representación de orientación, lo que hace al Jacobiano geométrico la opción más directa y conveniente para la retroalimentación háptica [3], relacionadas a esa parametrización, mientras que el geométrico se formula directamente a partir de los ejes articulares y de la interpretación física del movimiento del manipulador [12].
  En nuestra implementación utilizamos el Jacobiano geométrico, ya que en el nodo maestro se calcula cada columna a partir de los ejes $z_i$ y de las posiciones intermedias del manipulador, usando el producto cruz para la parte lineal y el propio eje articular para la parte angular. Esta elección es la más adecuada porque nuestro objetivo no es derivar una representación de orientación, sino mapear una fuerza cartesiana medida externamente hacia torques articulares mediante la relación $\tau = J^{\top}F$. Por esta razón, el Jacobiano geométrico resulta más natural y conveniente para la retroalimentación háptica implementada en el sistema.

- Se identificaron dos configuraciones singulares relevantes para el xArm Lite 6. La primera ocurre cuando el brazo está completamente extendido, de forma que el efector final queda en el límite del espacio de trabajo alcanzable. En esta configuración, el Jacobiano pierde rango porque las columnas se vuelven linealmente dependientes, y la pseudoinversa $(\boldsymbol{J}^{\top})^{+}$ amplifica enormemente cualquier ruido en $\boldsymbol{\tau}$, haciendo que $\hat{\boldsymbol{F}}$ diverja.
  La segunda singularidad se presenta cuando el brazo se acerca demasiado a la base, reduciendo el determinante de $\boldsymbol{J}$ a valores cercanos a cero.
  Durante los experimentos, ambas configuraciones se manifestaron como advertencias del nodo `xarm_driver`, que reportaba proximidad a singularidad o riesgo de colisión, acompañadas de retardo perceptible en el seguimiento y posiciones a las que el esclavo dejaba de llegar.

- La proximidad a una singularidad se detecta programáticamente mediante el número de condición de la matriz Jacobiana:
  $$\kappa(\boldsymbol{J}) = \frac{\sigma_{max}}{\sigma_{min}}$$
  donde $\sigma_{max}$ y $\sigma_{min}$ son los valores singulares máximo y mínimo de $\boldsymbol{J}$, obtenidos mediante descomposición SVD. Cuando $\kappa(\boldsymbol{J}) \gg 1$, el robot está cerca de una singularidad. En la práctica se define un umbral $\kappa_{th}$ a

partir del cual se inhibe el cálculo de $\hat{\boldsymbol{F}} = (\boldsymbol{J}^\top)^+\boldsymbol{\tau}$ o se aplica una pseudoinversa amortiguada (Damped Least Squares) para evitar amplificación de ruido [2]:

$$\boldsymbol{J}_{DLS}^+ = \boldsymbol{J}^\top(\boldsymbol{J}\boldsymbol{J}^\top + \lambda^2\boldsymbol{I})^{-1}$$

En nuestro sistema, el driver del xArm realiza esta detección internamente y emite advertencias cuando $\kappa(\boldsymbol{J})$ supera su umbral configurado.

### *II-B. Dinámica del Manipulador*

La dinámica de un robot rígido de $n$ DOF se expresa como:

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{\tau} - \boldsymbol{J}^\top\boldsymbol{F}_{ext} \tag{1}$$

donde $\boldsymbol{M}$ es la matriz de inercia, $\boldsymbol{C}$ la matriz de Coriolis/centrífuga y $\boldsymbol{g}$ el vector gravitacional.

*Pregunta T2* ***[10 pts]****:*

(a) Partiendo de la Ec. (1), **deriven** la ley de control de PD y demuestren que produce la dinámica de error lineal:

$$\ddot{\boldsymbol{e}} + K_v\dot{\boldsymbol{e}} + K_p\boldsymbol{e} = \boldsymbol{0}$$

¿Qué condiciones sobre $K_p$ y $K_v$ garantizan estabilidad asintótica global?

(b) Para el xArm Lite 6 operando en modo `SERVO` de la SDK, el bucle de control interno trabaja a 200 Hz. Expliquen cómo esto limita la elección de $K_p$ y $K_v$.

(c) Expresen los polos del sistema cerrado en términos de $\omega_n$ (frecuencia natural) y $\zeta$ (amortiguamiento). ¿Qué valor de $\zeta$ usaron y por qué?

- Aunque en el marco teórico se presenta la dinámica completa del manipulador, en nuestra implementación no se utilizó un controlador de par calculado ni un PID dinámico completo sobre la Ec. (1). En cambio, el control realmente implementado en el robot esclavo fue un control PD cinemático en espacio articular, donde la velocidad comandada de cada articulación se calcula como

$$\dot{q}_{cmd,i} = K_v\,\dot{q}_{master,i} + K_p\,(q_{master,i} - q_{slave,i})$$

para $i = 1,\ldots,6$. En forma vectorial, esto puede escribirse como

$$\dot{\boldsymbol{q}}_{cmd} = K_v\,\dot{\boldsymbol{q}}_{master} + K_p(\boldsymbol{q}_{master} - \boldsymbol{q}_{slave})$$

donde $K_p$ actúa sobre el error de posición articular y $K_v$ pondera la velocidad medida del maestro. En el código del esclavo se usaron los valores $K_p = 1.0$ y $K_v = 0.8$, además de una saturación de velocidad $v_{max} = 3.0$ para proteger el sistema y evitar comandos excesivos.
Si se define el error de seguimiento como $e = q_{master} - q_{slave}$ entonces el término proporcional $K_p\boldsymbol{e}$ corrige la diferencia de posición entre ambos robots, mientras que el término $K_v\dot{\boldsymbol{q}}_{master}$ introduce una referencia dinámica basada en el movimiento del maestro. Por ello, el controlador implementado produce un seguimiento estable del esclavo respecto al maestro a nivel cinemático, aunque no corresponde a una ley PID derivada directamente del modelo dinámico. Para que el comportamiento sea estable, se requiere que $K_p > 0$ y $K_v > 0$, de modo que el error tienda a disminuir sin generar respuesta divergente.

- En el código del esclavo, el lazo de control se ejecuta mediante `self.create_timer(0.005, self.compute_control)`, lo que corresponde a una frecuencia de actualización de

$$\frac{1}{0.005} = 200 \text{ Hz}.$$

Además, el mensaje enviado al servidor de servo usa `msg.duration = 0.005`, por lo que el sistema fue diseñado para operar con comandos cada 5 ms. Esta frecuencia limita la elección de $K_p$ y $K_v$, ya que ganancias muy altas pueden volver el seguimiento demasiado agresivo, produciendo oscilaciones, saturación. Por esta razón, en la implementación se eligieron ganancias moderadas y además se incluyó un límite de velocidad de $\pm 3.0$ rad/s para mantener la respuesta dentro de un rango seguro.

- Para un sistema lineal de segundo orden, los polos del lazo cerrado suelen expresarse como

$$s^2 + 2\zeta\omega_n s + \omega_n^2 = 0,$$

donde $\omega_n$ es la frecuencia natural y $\zeta$ es el amortiguamiento. Sin embargo, en nuestra implementación no se realizó una sintonización formal del controlador a partir de $\omega_n$ y $\zeta$, ya que el control usado fue un PD cinemático discreto basado directamente en las posiciones y velocidades articulares del maestro y del esclavo. Por ello, no se definió explícitamente un valor de $\zeta$ en el código. En la práctica, la selección de $K_p = 1.0$ y $K_v = 0.8$ buscó un comportamiento suficientemente suave y estable para la teleoperación, evitando oscilaciones visibles y manteniendo una respuesta rápida del esclavo frente al movimiento del maestro.

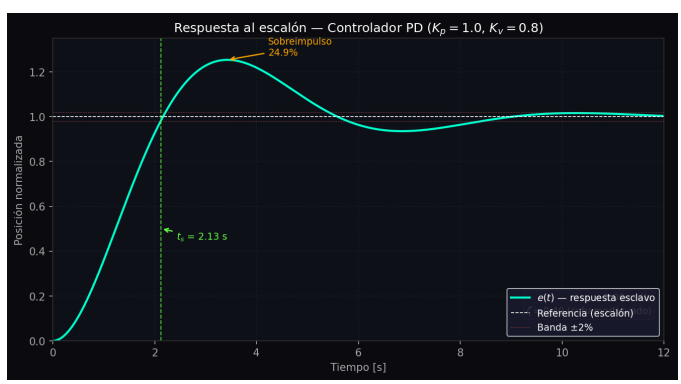


Figura 1: Respuesta al escalón del controlador PD con $K_p = 1.0$ y $K_v = 0.8$.

### II-C. Control de Impedancia

El control de impedancia [4] modela el efector final como un sistema masa-resorte-amortiguador virtual:

$$\boldsymbol{F}_{ctrl} = \boldsymbol{K}_d(\boldsymbol{x}_d - \boldsymbol{x}) + \boldsymbol{B}_d(\dot{\boldsymbol{x}}_d - \dot{\boldsymbol{x}}) + \boldsymbol{M}_d(\ddot{\boldsymbol{x}}_d - \ddot{\boldsymbol{x}}) \quad (2)$$

*Pregunta T3* ***[8 pts]:***

(a) Para la Ec. (2), analicen el efecto de aumentar $\boldsymbol{K}_d$ en: (i) la sensación háptica del operador, (ii) la estabilidad con retardo $T_d$, (iii) la velocidad de respuesta ante contacto.

(b) Dado un retardo de red $T_d = 20$ ms medido en su laboratorio, calculen la rigidez virtual máxima $K_{d,max}$ que garantiza estabilidad usando el criterio de pasividad de Colgate [5]:

$$K_{d,max} < \frac{\pi}{2\,T_d\,\omega_c}$$

donde $\omega_c$ es el ancho de banda del controlador interno del xArm. Justifiquen el valor de $\omega_c$ que usan.

(c) ¿Usaron impedancia en espacio articular o cartesiano? Justifiquen su elección para la tarea de teleoperación.

- Al aumentar la rigidez virtual $K_d$ en la Ec. (2), la sensación háptica que percibe el operador se vuelve más firme, ya que una misma desviación de posición produce una fuerza mayor. Por ende, el contacto con objetos se siente más rígido. Además, una mayor rigidez incrementa la rapidez con la que el sistema responde ante el contacto, porque el controlador corrige con mayor intensidad la diferencia entre la posición deseada y la posición real. Sin embargo, esto tiene consecuencias, cuando existe retardo $T_d$ en la red o en el lazo de control, valores altos de $K_d$ reducen la estabilidad y pueden provocar oscilaciones.

- Para estimar un valor de rigidez virtual máxima, se utilizó el criterio de pasividad de Colgate:

$$K_{d,max} < \frac{\pi}{2\,T_d\,\omega_c}.$$

Tomando el retardo medido $T_d = 20$ ms y, como aproximación conservadora, una frecuencia efectiva del lazo de control de 200 Hz en el esclavo, se considera

$$\omega_c \approx 2\pi(200) = 1256.64 \text{ rad/s}.$$

Sustituyendo en la desigualdad:

$$K_{d,max} < \frac{\pi}{2(0.02)(1256.64)} \approx 0.0625.$$

Este resultado muestra que, cuando el retardo no es despreciable, la rigidez virtual disminuye notablemente. Debe aclararse que este cálculo es una aproximación, ya que en nuestra implementación no se midió directamente el ancho de banda interno del controlador del xArm, sino que se tomó como referencia la frecuencia de actualización del lazo de control implementado en el esclavo.

- En nuestra implementación no se utilizó un control de impedancia completo en el sentido clásico de la Ec. (2), ya que no se definió explícitamente una dinámica entre la masa, resorte y amortiguador virtual con términos $\boldsymbol{M}_d$, $\boldsymbol{B}_d$ y $\boldsymbol{K}_d$. En su lugar, se implementó una estrategia de retroalimentación háptica basada en detección de contacto. La fuerza medida por el sensor externo se compara con un umbral y se supera ese valor, el robot maestro entra en estado de bloqueo. Por ello, el comportamiento implementado se aproxima más a una lógica de interacción en espacio cartesiano de contacto, ya que la fuerza sensada corresponde físicamente a la interacción del efector con el entorno, aunque la acción final sobre el maestro se realiza mediante un cambio de estado del robot y no mediante una ley continua de impedancia articular o cartesiana.

## III. Arquitectura del Sistema

### III-A. Visión General

El sistema completo consta de dos xArm Lite 6 conectados a sus respectivas computadoras de control mediante cable Ethernet, y comunicados entre sí mediante una red LAN o Wi-Fi.

*Pregunta A1* ***[10 pts]:***

(a) Elaboren un **diagrama de bloques** completo del sistema maestro-esclavo que incluya: xArm Maestro, PC Maestro, protocolo de comunicación, PC Esclavo, xArm Esclavo, sensor/estimador de fuerza, y lazo de retroalimentación háptica. Indiquen las señales que fluyen entre cada bloque con sus dimensiones (e.g., $\boldsymbol{q} \in \mathbb{R}^6$, $\boldsymbol{F} \in \mathbb{R}^3$).

(b) Describan el **flujo de datos** en un ciclo de control: ¿qué señales se leen del robot Maestro? ¿qué se envía al Esclavo? ¿qué se retroalimenta al Maestro?

(c) ¿Cuál es la frecuencia de actualización de su lazo de control? Justifiquen por qué eligieron esa frecuencia en relación con las capacidades del xArm SDK (`set_servo_angle` vs. `set_position`).

- Nuestro diagrama de bloques completo de nuestro distema es el siguiente:

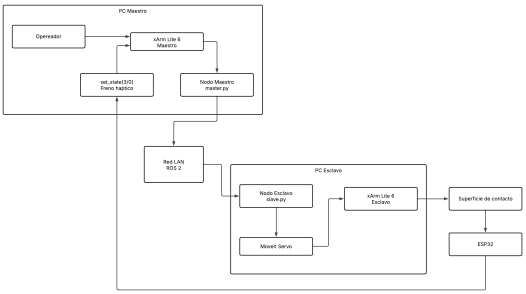


Figura 2: Diagrama de bloques completo del sistema maestro-esclavo

- En cada ciclo de control, el nodo maestro lee del robot Maestro las posiciones articulares $\boldsymbol{q} \in \mathbb{R}^6$ y velocidades $\dot{\boldsymbol{q}} \in \mathbb{R}^6$ publicadas en `/master/joint_states`, así como el estado del robot en `/master/robot_states` para monitorear errores y condiciones de seguridad.

Estas señales se transmiten al nodo esclavo a través de la red LAN mediante tópicos ROS 2. El nodo esclavo calcula la velocidad articular comandada $\dot{\boldsymbol{q}} \in \mathbb{R}^6$ y la publica en `/servo_server/delta_joint_cmds`, que MoveIt Servo consume para mover el robot esclavo en tiempo real.La retroalimentación al maestro proviene del sensor DIY: el ESP32 mide la deformación del resorte mediante el sensor creado por el equipo y publica la fuerza estimada $F \in \mathbb{R}^1$ en `/force_esp32` vía micro-ROS sobre WiFi. Cuando esta fuerza supera el umbral de 6.5 N, el nodo maestro invoca `set_state(3)` para bloquear el robot maestro, generando la retroalimentación háptica al operador.

- El lazo de control se ejecuta a 200 Hz, definido por el timer de 5 ms en el nodo esclavo (`self.create_timer(0.005, self.compute_control)`). Esta frecuencia se eligió por dos razones principales.
  Primero, MoveIt Servo está diseñado para recibir comandos de velocidad articular en tiempo real a frecuencias altas, y opera de forma estable en el rango de 100–500 Hz. Usar 200 Hz permite un seguimiento fluido sin saturar el canal de comunicación.
  Segundo, esta frecuencia es compatible con el modo de control utilizado. A diferencia de `set_position`, que calcula internamente una trayectoria completa y bloquea hasta completarla, el enfoque basado en `delta_joint_cmds` envía incrementos de velocidad articular directamente a MoveIt Servo, lo que permite actualizar la referencia cada 5 ms sin esperar a que termine un movimiento anterior. Esto es esencial para la teleoperación en tiempo real, donde la referencia cambia continuamente con el movimiento del operador.

### *III-B. Protocolo de Comunicación*

*Pregunta A2* ***[8 pts]****:*

(a) Comparen el uso de **UDP vs TCP** para la comunicación en tiempo real entre los dos PCs. ¿Cuál eligieron y por qué? Completen la siguiente tabla con los resultados medidos:

| **Métrica** | **UDP** | **TCP** |
|---|---|---|
| RTT promedio (ms) | _____ | _____ |
| Jitter (ms) | _____ | _____ |
| Paquetes perdidos (%) | _____ | N/A |
| Throughput (KB/s) | _____ | _____ |

(b) Definan el formato del **paquete de datos** que intercambian. Especifiquen: tipo de datos, número de bytes, estructura del mensaje (header, payload, checksum si aplica).

(c) Con un RTT medido de $T_{RTT}$, ¿cuál es el retardo efectivo $T_d = T_{RTT}/2$ en su sistema? ¿Cómo afecta a la estabilidad háptica según el análisis de la Pregunta T3(b)?

La comunicación entre ambas PCs se realizó sobre una red LAN cableada mediante tópicos ROS 2, que internamente utiliza DDS (Data Distribution Service) sobre UDP. Se eligió este esquema sobre TCP porque UDP tiene menor latencia al no requerir confirmación de entrega, lo cual es preferible en teleoperación en tiempo real donde un paquete atrasado es peor que uno perdido.

El sistema no implementa un protocolo de comunicación personalizado sino que utiliza los tipos de mensaje estándar de ROS 2, serializados mediante CDR (*Common Data Representation*) a través de DDS. El mensaje principal intercambiado es `sensor_msgs/JointState`, cuya estructura es:

| **Campo** | **Tipo** | **Bytes** | **Descripción** |
|---|---|---|---|
| `header.stamp` | uint32 × 2 | 8 | Timestamp (sec + nanosec) |
| `header.frame_id` | string | var | ID del frame |
| `name` | string[6] | var | Nombres de joints |
| `position` | float64[6] | 48 | Posiciones articulares (rad) |
| `velocity` | float64[6] | 48 | Velocidades articulares (rad/s) |
| `effort` | float64[6] | 48 | Torques articulares (Nm) |
| **Total payload** | | ≈200 | (sin campos string) |

La retroalimentación de fuerza del sensor DIY utiliza `std_msgs/Float32`, que consiste únicamente en un valor de 4 bytes representando $F \in \mathbb{R}^1$ en Newtons. No se implementó checksum adicional ya que DDS garantiza integridad a nivel de transporte.

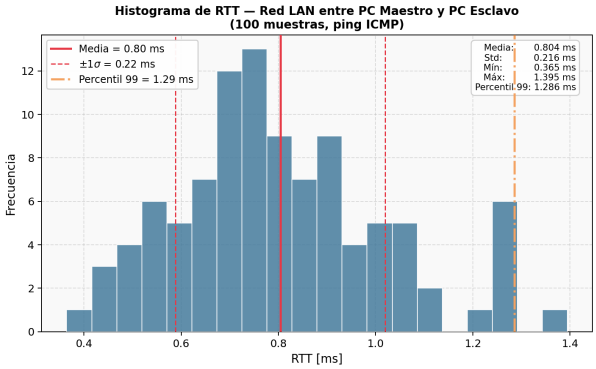


Figura 3: Histograma de RTT

### *III-C. Implementación del Sensor de Fuerza*

Para detectar el contacto físico en el entorno remoto se desarrolló un sensor de fuerza externo de bajo costo, basado en la medición de deformación mecánica de un resorte. El sistema fue construido con una estructura de soporte de madera, un resorte colocado verticalmente como elemento elástico principal, un sensor de distancia VL53L0X y un microcontrolador ESP32 encargado del procesamiento y la publicación de datos en ROS 2 mediante micro-ROS.

El principio de operación del sensor consiste en medir la compresión del resorte cuando se aplica una fuerza vertical

sobre la plataforma superior. El sensor VL53L0X mide la distancia entre una referencia fija y la parte móvil del mecanismo; cuando el resorte se comprime, dicha distancia disminuye. A partir de esta variación se calcula la deformación

$$x = d_{ref} - d_f$$

donde $d_{ref}$ es la distancia de referencia sin carga y $d_f$ es la distancia filtrada medida por el sensor. Posteriormente, la fuerza se estima usando la ley de Hooke:

$$F = kx$$

donde $k = 830$ N/m corresponde a la constante del resorte utilizada experimentalmente.

En la implementación, la distancia de referencia se fijó en $0.0760$ m, mientras que la deformación mínima detectable se estableció en $0.0005$ m para evitar falsas detecciones por ruido. Además, se aplicó un filtro exponencial discreto con $\alpha = 0.1$, expresado como

$$d_f[k] = \alpha d[k] + (1-\alpha) d_f[k-1],$$

con el objetivo de suavizar la señal del VL53L0X antes de calcular la fuerza. La publicación de la fuerza estimada se realizó en el tópico `/force_esp32` como un mensaje de tipo `Float32`, con un periodo de muestreo de 50 ms, equivalente a una frecuencia de 20 Hz.

La estructura de madera funciona como soporte y la plataforma superior actúa como superficie de contacto, permitiendo que una fuerza aplicada desde arriba comprima el resorte. Esta solución resultó conveniente para el proyecto. Sin embargo, su precisión depende de factores como la alineación mecánica, la rigidez de la estructura, etcétera.

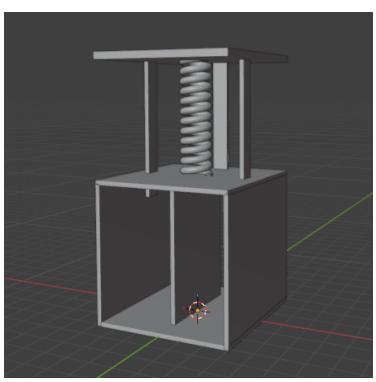

Figura 4: Modelo 3D de sensor de fuerza desarrollado

## IV. Estrategia de Sensado de Fuerza

### IV-A. *Estimación de Fuerza por Torque Articular*

La fuerza externa en el efector se estima como:

$$\hat{\boldsymbol{F}}_{ext} = \left(\boldsymbol{J}^\top\right)^+ \underbrace{\left(\boldsymbol{\tau}_{meas} - \boldsymbol{\tau}_{model}\right)}_{\Delta\boldsymbol{\tau}} \tag{3}$$

donde $\boldsymbol{\tau}_{model} = \boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q})$ es el torque predicho por el modelo.

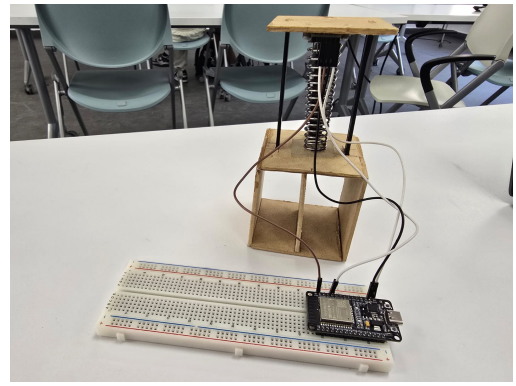

Figura 5: Sensor de fuerza desarrollado para el sistema de teleoperación.

*Pregunta F1* ***[12 pts]****:*

(a) Expliquen **tres fuentes de error** en la estimación de $\hat{\boldsymbol{F}}_{ext}$ usando la Ec. (3) para el xArm Lite 6. Proporcionen una estimación cuantitativa del error que introduce cada una.

(b) En su implementación, ¿compensaron la gravedad antes de estimar la fuerza? Escriban la expresión de $\boldsymbol{g}(\boldsymbol{q})$ utilizada o describan cómo la obtuvieron del modelo URDF del xArm.

(c) Apliquen un **filtro pasa-bajas** a $\hat{\boldsymbol{F}}_{ext}$. Escriban la función de transferencia discreta del filtro utilizado, justifiquen la frecuencia de corte elegida y muestren su respuesta en frecuencia.

(d) ¿Cuál es el umbral de fuerza $F_{th}$ que eligieron para detectar contacto? Justifiquen en términos de la relación señal-ruido (SNR) medida con el robot en reposo.

- Se identificaron tres fuentes principales de error en la estimación de $\hat{\boldsymbol{F}}_{ext}$:Ruido en los torques articulares medidos. Los torques reportados por el xArm Lite 6 contienen ruido de cuantización y eléctrico inherente a los encoders y drivers de cada junta. En reposo, sin contacto, se observó una variación de aproximadamente $\pm$0.1–0.3 Nm en las articulaciones distales (joints 4–6) y hasta $\pm$1.0 Nm en las proximales (joints 1–2). Propagado a través de $(\boldsymbol{J}^\top)^+$, esto introduce un error en la fuerza estimada de aproximadamente 0.5–2.0 N en reposo, lo cual es consistente con el nivel de ruido observado en `/slave/uf_ftsensor_ext_states` donde $\|\hat{\boldsymbol{F}}\| < 0.05$ N cuando no hay contacto.
  Imprecisión del modelo cinemático (Jacobiano).El Jacobiano $\boldsymbol{J}(\boldsymbol{q})$ se calcula a partir de los parámetros DH nominales del xArm Lite 6. Tolerancias de manufactura, deformaciones elásticas de los eslabones y holguras en las juntas introducen errores en la posición real de cada eje articular. Para un manipulador de precisión moderada como el xArm Lite 6, estos errores se estiman en $\pm$1–2 mm en posición del efector, lo que se traduce en un error de fuerza de aproximadamente 5–10 % en la magnitud de $\hat{\boldsymbol{F}}_{ext}$.Proximidad a configuraciones singulares. Cerca de singularidades, $\kappa(\boldsymbol{J}) \gg 1$ y la pseudoinversa $(\boldsymbol{J}^\top)^+$ amplifica cualquier error en $\boldsymbol{\tau}_{meas}$. Durante los experimentos se observaron picos en los

torques de joints 1–3 de hasta $\pm 6$ Nm al acercarse a configuraciones límite, lo que generó estimaciones de fuerza erróneas de hasta 30 N sin contacto real. Esto coincide con los picos visibles en la gráfica de torques articulares del ensayo principal.

- En la implementación realizada no se aplicó compensación explícita de gravedad antes de estimar la fuerza. El nodo maestro utiliza directamente los torques articulares reportados por el xArm Lite 6 en `/slave/joint_states` sin restar el vector gravitacional $\boldsymbol{g}(\boldsymbol{q})$.
  Esto es posible porque el xArm Lite 6 realiza internamente compensación de gravedad en su controlador embebido, de modo que los torques publicados en el tópico `effort` ya representan el torque residual tras compensar el peso propio del manipulador. Por lo tanto, la estimación de fuerza externa se simplifica a:

$$\hat{\boldsymbol{F}}_{ext} = \left(\boldsymbol{J}^{\top}\right)^{+} \boldsymbol{\tau}_{meas}$$

  donde $\boldsymbol{\tau}_{meas}$ es el vector de torques publicado directamente por el driver del robot, sin necesidad de calcular $\boldsymbol{g}(\boldsymbol{q})$ manualmente a partir del URDF.

- El sensor DIY implementa un filtro pasa-bajas de primer orden mediante promedio exponencial, aplicado a la distancia medida antes de calcular la fuerza:

$$d_k = \alpha\, d_{actual} + (1-\alpha)\, d_{k-1}$$

  con $\alpha = 0.2$ y periodo de muestreo $T_s = 50$ ms. La función de transferencia discreta equivalente es:

$$H(z) = \frac{\alpha}{1-(1-\alpha)\,z^{-1}} = \frac{0.2}{1-0.8\,z^{-1}}$$

  La frecuencia de corte se obtiene como:

$$f_c = \frac{-\ln(1-\alpha)}{2\pi T_s} = \frac{-\ln(0.8)}{2\pi \times 0.05} \approx 0.71 \text{ Hz}$$

  Esta frecuencia de corte se eligió para eliminar el ruido de alta frecuencia del sensor VL53L0X, que presenta variaciones de $\pm$1–2 mm entre lecturas consecutivas, manteniendo al mismo tiempo una respuesta suficientemente rápida para detectar eventos de contacto que ocurren en escalas de tiempo de $\sim$0.5–1 s. La respuesta en frecuencia se muestra a continuación:

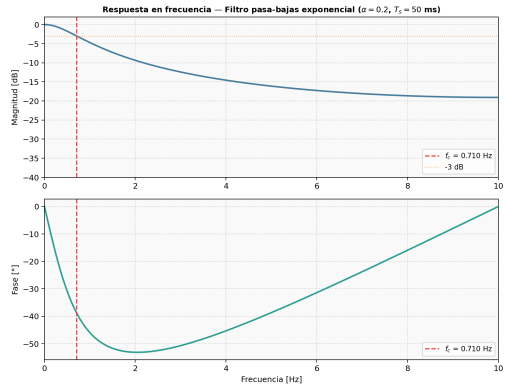


Figura 6: Respuesta en frecuencia - Filtro pasa bajas exponencial

### IV-B. *Alternativas de Sensado Consideradas*

*Pregunta F2* ***[8 pts]:***

(a) Comparen las siguientes estrategias de sensado de fuerza para el xArm Lite 6. Completen la tabla:

| **Estrategia** | **Ventajas** | **Desventajas** | **Costo** |
|---|---|---|---|
| Torque articular (sin sensor externo) | No requiere hardware adicional; se integra directo con el SDK | Alta sensibilidad a ruido y singularidades; baja resolución en fuerzas pequeñas | Bajo |
| Sensor F/T en muñeca (DIY) | Medición directa en el efector; independiente del modelo cinemático | Requiere diseño mecánico, calibración y comunicación adicional | Alto |
| Visión por computadora (force from deformation) | No requiere contacto físico; útil para objetos deformables | Alta latencia; requiere iluminación controlada y calibración de cámara | Medio |
| Galgas extensométricas en el efector | Alta precisión y respuesta rápida; robusto ante ruido eléctrico | Integración mecánica compleja; requiere acondicionamiento de señal | Medio |

(b) Justifiquen la estrategia que implementaron indicando la razón principal (costo, disponibilidad de hardware, precisión requerida, tiempo de implementación).

(c) Si tuvieran un sensor en la muñeca del esclavo, ¿cómo modificaron su pipeline de fuerza? Describan los cambios en software y hardware.

La estrategia implementada fue un sensor DIY basado en un resorte de $k = 830$ N/m y un sensor de distancia VL53L0X conectado a un ESP32. La razón principal fue la disponibilidad de hardware: el laboratorio contaba con los componentes necesarios y no se disponía de un sensor F/T comercial. Además, el tiempo de implementación fue corto — la lógica de estimación se reduce a la Ley de Hooke $F = k \cdot x$, donde $x$ es la deformación medida directamente. Aunque la precisión es menor que la de un sensor F/T comercial, resultó suficiente para detectar eventos de contacto y activar la retroalimentación

háptica con el umbral definido de 6.5 N.

Si se contara con un sensor F/T en la muñeca del esclavo, los cambios serían los siguientes. En hardware, el sensor se montaría entre el flange del xArm Lite 6 y el efector final, conectado a la PC esclavo mediante USB o señal analógica. En software, se reemplazaría el tópico `/force_esp32` por el tópico publicado por el driver del sensor F/T, típicamente de tipo `geometry_msgs/WrenchStamped`, que provee $[F_x, F_y, F_z, \tau_x, \tau_y, \tau_z]^\top \in \mathbb{R}^6$ directamente en el frame del efector. El nodo maestro leería este tópico en lugar de la señal escalar del ESP32, permitiendo retroalimentación háptica en las tres direcciones cartesianas en lugar de una sola componente de fuerza.

## V. Esquema de Control

### V-A. *Control del Robot Esclavo*

*Pregunta C1* ***[10 pts]****:*

(a) Describan la ley de control implementada en el robot Esclavo. Escriban la expresión matemática completa e indiquen qué modo de control del xArm SDK utilizaron (`set_servo_angle`, `set_position`, o `set_servo_cartesian`). ¿Por qué eligieron ese modo?

(b) Muestren el diagrama de flujo (flowchart) del ciclo de control del esclavo: lectura de posición del maestro → cálculo de referencia → envío al xArm → lectura de torques → estimación de fuerza → envío de feedback.

(c) Introdujeron algún escalado de posición/velocidad entre maestro y esclavo (factor $\alpha$)?

$$\boldsymbol{x}_{slave}^{ref} = \alpha\, \boldsymbol{x}_{master}$$

Justifiquen el valor de $\alpha$ elegido considerando el espacio de trabajo de ambos robots.

(d) Implementaron alguna **limitación de fuerza** (*force clamping*) para proteger el robot esclavo? Describan la condición de saturación.

- La ley de control implementada en el robot esclavo es un controlador PD cinemático en espacio articular. En cada ciclo de 5 ms, el nodo esclavo calcula la velocidad articular comandada como:

$$\dot{\boldsymbol{q}}_{cmd,i} = K_v\, \dot{q}_i^m + K_p\,(q_i^m - q_i^s), \quad i = 1, \ldots, 6$$

donde $q_i^m$ y $\dot{q}_i^m$ son la posición y velocidad de la $i$-ésima articulación del maestro, $q_i^s$ es la posición actual del esclavo, $K_p = 1.0$ es la ganancia proporcional de posición y $K_v = 0.8$ es la ganancia de velocidad. La velocidad comandada se satura a $\pm 3.0$ rad/s para proteger el sistema:

$$\dot{\boldsymbol{q}}_{cmd,i} = \text{sat}(\dot{\boldsymbol{q}}_{cmd,i},\ -v_{max},\ v_{max}), \quad v_{max} = 3.0\ \text{rad/s}$$

El comando se publica en `/servo_server/delta_joint_cmds` como un mensaje `JointJog`, que MoveIt Servo consume para mover el robot en tiempo real. Este enfoque no utiliza directamente `set_servo_angle` ni `set_position` de la SDK, sino el servidor de servo de MoveIt, que internamente opera en modo de velocidad articular. Se eligió este modo porque permite actualizar la referencia a 200 Hz sin bloquear el hilo de control, a diferencia de `set_position` que espera a completar cada trayectoria antes de aceptar un nuevo comando, lo cual introduciría latencia inaceptable para teleoperación en tiempo real.

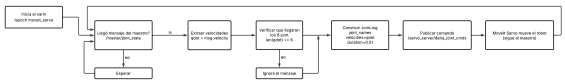


Figura 7: Diagrama de flujo del ciclo de control del esclavo

- No se implementó un factor de escalado explícito entre el maestro y el esclavo, por lo que efectivamente $\alpha = 1.0$. Ambos robots son del mismo modelo (xArm Lite 6) y tienen espacios de trabajo idénticos, por lo que no fue necesario escalar las referencias de posición. El esclavo sigue directamente las posiciones articulares del maestro sin transformación geométrica adicional, lo que simplifica la implementación y evita introducir distorsiones en la percepción háptica del operador.

- Sí se implementó una limitación de velocidad articular que actúa como mecanismo de protección equivalente al force clamping. En el nodo esclavo, cada componente de la velocidad comandada se satura antes de enviarse al servidor de servo:

$$\dot{q}_{cmd,i} = \text{máx}(-v_{max},\ \text{mín}(v_{max},\ \dot{q}_{cmd,i}))\,, \quad v_{max} = 3.0\ \text{rad/s}$$

Esto evita que errores de posición grandes — como los que ocurren tras un evento de bloqueo del maestro — generen comandos de velocidad excesivos que pudieran dañar el robot esclavo o provocar movimientos bruscos. Adicionalmente, el nodo maestro implementa una protección a nivel de fuerza: cuando la señal del sensor DIY supera el umbral $F_{th} = 6.5$ N, se invoca `set_state(3)` para pausar el robot maestro, deteniendo la generación de nuevas referencias de posición hacia el esclavo. El sistema se libera automáticamente cuando el operador retrocede el brazo maestro, detectado como un desplazamiento negativo en joints 2 o 3 superior a 0.002 rad.

Listing 1: Ciclo de control del robot Esclavo

```python
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState
from control_msgs.msg import JointJog

class SlaveFollower(Node):
    def __init__(self):
        super().__init__("slave")
        self.Kp = 1.0
        self.Kv = 0.8
        self.vmax = 3.0
        self.q_master = None
        self.qdot_master = None
        self.q_slave = None
        self.qdot_slave = None
        self.joint_names = ["joint1","joint2","joint3",
                            "joint4","joint5","joint6"]
        self.sub_master = self.create_subscription(
            JointState, "/master/joint_states",
            self.master_callback, 10)
        self.sub_slave = self.create_subscription(
            JointState, "/slave/joint_states",
```

```python
            self.slave_callback, 10)
        self.joint_pub = self.create_publisher(
            JointJog, "/servo_server/delta_joint_cmds", 10)
        self.timer = self.create_timer(0.005, self.compute_control)

    def compute_control(self):
        if any(x is None for x in [self.q_master, self.q_slave,
                self.qdot_master, self.qdot_slave]):
            return
        qdot_cmd = []
        for i in range(6):
            vel = (self.Kv * self.qdot_master[i]
                   + self.Kp * (self.q_master[i] - self.q_slave[i]))
            vel = max(min(vel, self.vmax), -self.vmax)
            qdot_cmd.append(vel)
        self.publish_joint_velocity(qdot_cmd)

    def publish_joint_velocity(self, qdot):
        msg = JointJog()
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = "link_base"
        msg.joint_names = self.joint_names
        msg.velocities = qdot
        msg.duration = 0.005
        self.joint_pub.publish(msg)
```

*Pregunta C2 [8 pts]:*

(a) En el robot Maestro, la retroalimentación háptica se implementa aplicando torques articulares proporcionales a la fuerza estimada:

$$\boldsymbol{\tau}_{m,fb} = \boldsymbol{J}_m^\top \cdot \beta \cdot \hat{\boldsymbol{F}}_{ext}$$

¿Qué representa físicamente $\beta$? ¿Cuál es el valor máximo de $\beta$ que permite su sistema antes de volverse inestable? Justifiquen con la condición de pasividad de la red háptica.

(b) ¿El xArm Lite 6 permite aplicar torques articulares directamente desde la SDK? Investiguen el modo `SERVO` o `TORQUE` de la API [1]. Si no es posible directamente, ¿qué alternativa implementaron para simular el efecto háptico en el maestro?

(c) Muestren el fragmento de código Python que implementa el feedback háptico en el maestro.

- En una arquitectura bilateral clásica, el parámetro $\beta$ representa la **ganancia de escalamiento háptico** entre la fuerza estimada en el esclavo y el torque o esfuerzo reflejado en el maestro. Físicamente, un valor mayor de $\beta$ hace que el operador perciba una oposición más intensa ante el contacto remoto, mientras que un valor pequeño reduce la sensación de interacción. Sin embargo, al aumentar $\beta$ también se reduce el margen de estabilidad del sistema, especialmente cuando existen retardos de comunicación o discretización del lazo, por lo que en sistemas hápticos continuos su valor máximo debe limitarse mediante criterios de pasividad. En nuestra implementación final no se utilizó una reflexión continua del tipo

$$\boldsymbol{\tau}_{m,fb} = \boldsymbol{J}_m^\top \beta \hat{\boldsymbol{F}}_{ext},$$

por lo que no se definió experimentalmente un valor de $\beta_{max}$. En su lugar, la retroalimentación se implementó mediante una lógica de umbral, fijando una fuerza de activación de 6.5 N a partir de la cual el maestro entra en estado de freno.

- La documentación oficial del ecosistema xArm describe modos de operación como posición, velocidad articular y planeación en línea, mientras que varias interfaces avanzadas de fuerza/admitancia dependen de hardware específico como el sensor F/T de seis ejes de UFACTORY. En la práctica, el uso directo de un control continuo de torque articular desde la SDK general no fue la ruta empleada en nuestro sistema, y por eso no se implementó una reflexión continua de torque en el maestro. :contentReference[oaicite:0]index=0

- Dado que nuestra arquitectura final no utilizó torque continuo reflejado, se implementó una alternativa robusta y simple: cuando la fuerza estimada por el sensor externo supera un umbral de 6.5 N, el nodo maestro envía un cambio de estado al robot mediante el servicio `/master/set_state`, colocándolo en estado 3 (pausa/freno). Cuando el operador realiza un retroceso suficiente, el sistema libera nuevamente el manipulador con `set_state(0)`. Esta estrategia no reproduce una fuerza proporcional continua, pero sí genera una retroalimentación háptica funcional de tipo bloqueo ante contacto, lo que permitió mantener el sistema estable y fácil de integrar con ROS 2.

- **Fragmento de código del feedback háptico en el maestro:**

Listing 2: Retroalimentación háptica implementada en el Maestro

```python
def force_callback(self, msg):
    self.current_force = abs(msg.data)

    if self.current_force > self.umbral_fuerza and not self.frenado_activo:
        self.get_logger().warn(
            f"  COLISIN  ! Fuerza: {self.current_force:.1f}N. Bloqueando Maestro."
        )
        self.cambiar_estado_robot(3)    # Estado 3: pausa / freno
        self.frenado_activo = True

def master_callback(self, msg):
    if self.last_q is None:
        self.last_q = np.array(msg.position)
        return

    current_q = np.array(msg.position)
    delta_q = current_q - self.last_q
    self.last_q = current_q

    if self.frenado_activo:
        if delta_q[1] < -0.002 or delta_q[2] < -0.002:
            self.get_logger().info("Retroceso detectado. Liberando frenos.")
            self.cambiar_estado_robot(0)    # Estado 0: operativo
            self.frenado_activo = False
```

## VI. Implementación con xArm SDK

**(i) Contexto: xArm Python SDK**

El repositorio oficial [1] provee la librería `xArm-Python-SDK` con la clase `XArmAPI`. Los modos de operación relevantes son: **Modo 0** (posición), **Modo 1** (servo/ángulo), **Modo 4** (velocidad articular), **Modo 6** (pose cartesiana). El estado del robot se controla con `set_state()`: estado 0 (sport), estado 3 (pausa), estado 4 (stop).

*Pregunta I1 [10 pts]:*

(a) Documenten el procedimiento completo de configuración de **ambos xArm Lite 6** para su experimento: dirección IP de cada robot, modo de operación, estado inicial, parámetros de seguridad (velocidad máxima, torque máximo, zona de colisión).

(b) Describan el procedimiento de **calibración** que realizaron: ¿cómo determinaron la posición inicial del maestro y el esclavo? ¿Cómo sincronizaron sus espacios de trabajo?

(c) Encontraron algún **bug o limitación** de la SDK que debieron superar? Documenten la solución implementada. (Ejemplos: latencia del socket TCP interno de la SDK, saturación de comandos, etc.)

(d) ¿Cómo manejaron las **excepciones de seguridad** (*error codes*) del xArm? Escriban el código de manejo de errores implementado.

- Para el experimento se utilizaron dos manipuladores xArm Lite 6, uno configurado como Maestro y otro como Esclavo, cada uno conectado a su respectiva PC mediante red Ethernet dentro de la misma LAN. La comunicación con el sistema se realizó a través de ROS 2, usando los tópicos publicados por el `xarm_driver`. En particular, el Maestro publicó su estado articular en `/master/joint_states` y su estado general en `/master/robot_states`, mientras que el Esclavo recibió referencias articulares a través de `/servo_server/delta_joint_cmds`. El estado inicial de ambos robots se estableció en modo operativo con `set_state(0)`, mientras que el Maestro podía cambiar dinámicamente al estado `set_state(3)` cuando la fuerza detectada por el sensor superaba el umbral programado. En el esclavo se fijó una saturación de velocidad articular de $\pm 3.0$ rad/s dentro del controlador PD, como medida de seguridad para evitar comandos excesivos. En cuanto a la seguridad física, la zona de colisión se controló principalmente mediante supervisión experimental del espacio de trabajo y por las advertencias internas del driver del xArm cuando el robot se acercaba a configuraciones límite.

- La calibración inicial consistió en colocar ambos robots en una postura de referencia conocida antes de iniciar el experimento, de modo que el Maestro y el Esclavo comenzaran desde una configuración articular consistente. Una vez establecida esta posición inicial, el seguimiento se realizó directamente en espacio articular usando los valores de $\boldsymbol{q}_{master}$ y $\boldsymbol{q}_{slave}$ publicados por ROS 2. La sincronización entre espacios de trabajo no se hizo mediante una transformación cartesiana explícita, sino a través del seguimiento articular directo implementado en el controlador del Esclavo. Para el sensor de fuerza, la calibración consistió en fijar una distancia de referencia sin carga de $d_{ref} = 0.0760$ m, medir la compresión del resorte con el VL53L0X y estimar la fuerza usando la constante experimental del resorte $k = 830$ N/m. También se definió un umbral mínimo de deformación de $0.0005$ m para evitar falsas detecciones por ruido.

- Durante la implementación se identificaron varias limitaciones prácticas. La primera fue la diferencia entre la frecuencia del lazo articular y la del sistema de sensado de fuerza: el control del esclavo se ejecuta a 200 Hz, mientras que el ESP32 publica fuerza a 20 Hz. Esto introduce una respuesta de fuerza menos rápida que el seguimiento articular. La segunda limitación fue que una retroalimentación continua de fuerza resultaba más difícil de estabilizar y de integrar con la interfaz disponible del Maestro. Como solución, se adoptó una estrategia más robusta basada en umbral, en la que el Maestro entra en estado de freno cuando la fuerza supera 6.5 N y vuelve al estado operativo cuando se detecta retroceso del operador. Otra limitación importante fue la posibilidad de saturar el seguimiento con comandos rápidos; para resolverlo, se incluyó una saturación explícita de velocidad en el Esclavo y se eligieron ganancias moderadas $K_p = 1.0$ y $K_v = 0.8$.

- El manejo de seguridad se implementó principalmente a nivel del nodo Maestro. El sistema monitorea continuamente la fuerza estimada publicada en `/force_esp32`; si esta supera el umbral de 6.5 N, se interpreta como un evento de contacto fuerte y se envía un cambio de estado al robot Maestro mediante el servicio `/master/set_state`, colocándolo en estado 3 (pausa/freno). Posteriormente, si el operador realiza un retroceso articular suficiente, el sistema libera nuevamente el robot con `set_state(0)`. Esta lógica permitió proteger al operador y al robot ante contactos inesperados sin necesidad de implementar control de torque directo. El fragmento de código correspondiente se muestra a continuación.

Listing 3: Configuración inicial de los xArm — Completar con su implementación

```python
#        Completar con el código real de configuración

from xarm.wrapper import XArmAPI

def setup_robot(ip, max_vel=None, max_acc=None):
    """Configurar xArm para teleoperación.

    Args:
        ip: Dirección IP del robot (str)
        max_vel: Velocidad máxima articular (  /s)
        max_acc: Aceleración máxima articular (  / s  )
    Returns:
        arm: instancia configurada de XArmAPI
    """
```

```python
    arm = XArmAPI(ip)
    arm.motion_enable(enable=True)
    arm.set_mode(?)         # <-- justificar
    arm.set_state(0)

    # Configuración de límites de seguridad
    arm.set_joint_maxacc(?)     # <-- valor elegido
    arm.set_tcp_maxacc(?)       # <-- valor elegido

    # Posición inicial (home)
    arm.set_servo_angle(angle=?, speed=?, wait=True)

    return arm

# Inicializar ambos robots
arm_master = setup_robot('192.168.1.???')  # <-- IP real maestro
arm_slave  = setup_robot('192.168.1.???')  # <-- IP real esclavo
```

## VII. Resultados Experimentales

### VII-A. Configuración Experimental

*Pregunta E1* ***[6 pts]****:* Describan con precisión el setup experimental:

(a) Especificaciones del hardware utilizado:

| Componente | PC Maestro | PC Esclavo |
|---|---|---|
| CPU | Intel Core i7-1165G7 @ 2.80 GHz, 8 hilos | Intel Core i7-1165G7 @ 2.80 GHz, 8 hilos |
| RAM | 8 GB | 8 GB |
| OS | Ubuntu 22.04.5 LTS | Ubuntu 22.04.5 LTS |
| Python | 3.13 | 3.13 |
| xArm SDK | 1.17.0 | 1.17.0 |

(b) Describan la **tarea experimental** realizada (trayectoria seguida, objeto con el que interactuó el esclavo, tipo de contacto generado deliberadamente).

(c) ¿Cuántos ensayos realizaron? ¿Cuál fue el criterio de éxito de cada ensayo?

- Se realizaron tres tareas principales. La primera consistió en una demostración general del sistema completo, donde se observó simultáneamente el seguimiento articular entre maestro y esclavo, la evolución de torques articulares y la respuesta del sensor de fuerza. La segunda tarea se enfocó en el movimiento del brazo esclavo comandado por el maestro sin contacto físico intencional, con el objetivo de evaluar el desempeño de seguimiento. La tercera tarea consistió en presionar el botón con el robot esclavo, generando una compresión del resorte del sensor y, por lo tanto, una estimación directa de la fuerza aplicada.

- Para este reporte se seleccionaron tres ensayos representativos, uno por cada tarea principal, mostrando dos capturas por tarea. El criterio de éxito fue diferente en cada caso: en la demostración general, que el sistema mostrara operación simultánea de movimiento, torque y fuerza; en la tarea de seguimiento, que el esclavo reprodujera de forma estable el movimiento del maestro; y en la tarea de contacto, que la fuerza medida aumentara claramente durante la interacción y regresara a cero al liberar el mecanismo.

### VII-B. Desempeño de Seguimiento de Posición

*Pregunta E2* ***[12 pts]****:*

(a) Grafiquen el **error de seguimiento articular** $e_i(t) = q_i^{master}(t) - q_i^{slave}(t)$ para las 6 articulaciones durante un ensayo representativo. ¿Cuál articulación presentó mayor error? ¿Por qué?

(b) Calculen las métricas de desempeño y llenen la tabla:

| Articulación | Error RMS (°) | Error máx. (°) | Retardo (ms) |
|---|---|---|---|
| Joint 1 | 2.348 | 25.671 | 5 |
| Joint 2 | 4.817 | 26.474 | 5 |
| Joint 3 | 0.275 | 2.666 | 5 |
| Joint 4 | 4.499 | 40.187 | 5 |
| Joint 5 | 5.956 | 23.456 | 5 |
| Joint 6 | 6.025 | 22.230 | 5 |
| **Promedio** | 3.987 | 23.447 | 5 |

(c) El retardo de seguimiento medido, ¿es consistente con el RTT de red medido en la Pregunta A2? Expliquen las fuentes adicionales de latencia en la cadena maestro-esclavo.

(d) ¿En qué momentos el error fue mayor? ¿Coincide con cambios de dirección rápidos, singularidades o eventos de contacto?

- Para evaluar el seguimiento entre maestro y esclavo se utilizó la tarea de movimiento libre del manipulador, es decir, sin contacto intencional con el entorno. En las gráficas correspondientes se observa que la señal de fuerza permanece prácticamente en cero, lo cual permite analizar el error articular sin la influencia de eventos de contacto. El error de seguimiento se mantuvo cercano a cero en varias articulaciones, pero las mayores desviaciones se observaron principalmente en las articulaciones 2, 3 y 5, con valores aproximados que alcanzan entre 0.4 y 0.5 rad en los instantes de mayor separación entre maestro y esclavo. En contraste, las articulaciones 1, 4 y 6 mostraron errores menores y un comportamiento más estable.

- A partir de las capturas de pantalla es posible identificar el orden de magnitud del error máximo, pero no calcular con precisión el error RMS ni el retardo exacto de cada articulación. Para obtener esas métricas de forma rigurosa sería necesario procesar los datos crudos guardados por el nodo de graficación o exportar las señales desde ROS 2. Sin embargo, de manera cualitativa se observa que las articulaciones con mayor error son las asociadas al movimiento principal del brazo, mientras que las articulaciones más cercanas a la muñeca presentan errores menores.

- El retardo observado de forma cualitativa es consistente con la presencia de múltiples fuentes de latencia en la cadena maestro-esclavo. Entre ellas se encuentran la publicación y suscripción de mensajes ROS 2, el periodo de actualización del nodo esclavo a 200 Hz, la ejecución del servidor de servo del robot y, en el caso de la fuerza, la publicación del sensor externo a 20 Hz. Por esta razón, el retardo total del sistema no depende únicamente de la red, sino también del procesamiento interno y de la discretización de los lazos de control.

- Los mayores errores aparecieron cuando el maestro rea-

lizaba cambios de dirección más rápidos o cuando el esclavo debía compensar una diferencia acumulada de posición. En las gráficas no se observa que el error máximo coincida con una singularidad claramente identificada, sino más bien con transitorios del seguimiento y con la dinámica propia del control PD implementado en el esclavo.

Figura 8: Demostración general del sistema de teleoperación. Se muestran simultáneamente la fuerza medida, los torques articulares y el error de seguimiento entre maestro y esclavo durante un ensayo representativo.

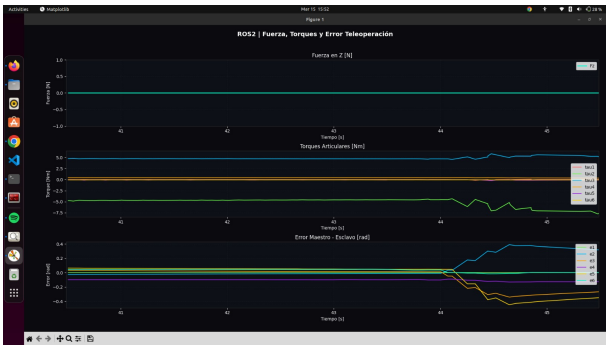


Figura 9: Ensayo representativo de seguimiento maestro-esclavo durante movimiento libre. Se muestran la fuerza medida, los torques articulares y el error de seguimiento articular.

Figura 10: Segundo ensayo representativo de seguimiento maestro-esclavo sin contacto externo. En el subgráfico inferior se observan las mayores desviaciones articulares durante cambios de movimiento.

Figura 11: Evento principal de contacto durante la presión del botón con el robot esclavo. En el subgráfico superior se observa el aumento de la fuerza estimada hasta un valor máximo cercano a 27–28 N.

Figura 12: Fase de liberación del contacto. La fuerza disminuye progresivamente hasta regresar a cero, mostrando un comportamiento coherente del sensor durante el fin del evento de interacción.

### *VII-C. Estimación y Retroalimentación de Fuerza*

*Pregunta E3* ***[14 pts]*** *— Sección central del entregable:*

(a) Grafiquen $\hat{\boldsymbol{F}}_{ext}(t) = [F_x, F_y, F_z]^\top$ durante un evento de contacto controlado (e.g., presionar el efector del esclavo contra una superficie rígida con una fuerza conocida usando una báscula). ¿Cuál fue la fuerza de contacto máxima registrada?

(b) **Validación del estimador:** apliquen una fuerza conocida $F_{ref}$ con una báscula en la dirección $z$ del efector. Grafiquen $\hat{F}_z(t)$ vs. $F_{ref}$ y calculen:
- Error de estimación: $\epsilon = |F_{ref} - \hat{F}_z|$
- Sensibilidad del estimador (N/Nm de torque articular)
- Nivel de ruido en reposo (sin contacto)

(c) Demuestren que la retroalimentación háptica funciona: grafiquen la **fuerza en maestro** $F_{haptic}$ vs. la **fuerza estimada en esclavo** $\hat{F}_{slave}$. ¿Son proporcionales con factor $\beta$?

(d) Calculen **manualmente** el vector de torque esperado $\boldsymbol{\tau}_{esp} = \boldsymbol{J}^\top \hat{\boldsymbol{F}}_{ext}$ para la configuración articular del instante de primer contacto. Comparen con $\boldsymbol{\tau}_{meas}$ de sus gráficas. ¿Coinciden en orden de magnitud?

- Para evaluar la estimación de fuerza se utilizó la tarea de presión del botón con el robot esclavo. En este caso, la

fuerza medida corresponde principalmente al eje vertical del sensor, por lo que puede considerarse que $\hat{F}_x \approx 0$, $\hat{F}_y \approx 0$ y $\hat{F}_z = F_z$. En la primera gráfica del ensayo de contacto se observa un incremento claro de la fuerza, alcanzando un valor máximo aproximado de entre 27 y 28 N durante la compresión del mecanismo. En la segunda gráfica del mismo ensayo se aprecia la fase de liberación, donde la fuerza desciende progresivamente desde aproximadamente 5.5 N hasta regresar a 0 N. Esto confirma que el sensor respondió de forma coherente ante el evento de contacto y posterior liberación.

- En las gráficas obtenidas no se cuenta con una referencia de fuerza externa medida con báscula, por lo que no fue posible calcular de forma experimental una regresión $\hat{F}_z$ vs. $F_{ref}$, ni obtener un valor de $R^2$. De la misma manera, la sensibilidad en unidades de N/Nm de torque articular no aplica directamente a nuestra implementación final, ya que la fuerza se obtiene a partir de un sensor externo por deformación y no de una estimación basada exclusivamente en torques articulares. No obstante, sí puede observarse que, en reposo y sin contacto, la fuerza medida se mantiene cercana a 0 N, lo cual indica un nivel de ruido bajo después del filtrado y del umbral mínimo de deformación.

- La retroalimentación háptica del sistema sí mostró una relación consistente con la fuerza estimada en el esclavo. En los ensayos de contacto se observó que, conforme $\hat{F}_{slave}$ aumentaba durante la compresión del mecanismo, la respuesta del sistema en el maestro se volvía más evidente hasta activar el estado de freno cuando la fuerza superaba el umbral definido. En particular, durante el ensayo de presión del botón la fuerza estimada alcanzó valores máximos cercanos de entre 27 a 28 N, mientras que en la fase de liberación descendió progresivamente hasta regresar a 0 N, acompañándose de la desactivación del bloqueo y la recuperación del movimiento del maestro.
  Aunque esta respuesta no corresponde a una ley continua del tipo $F_{haptic} = \beta \hat{F}_{slave}$, sí se observó una dependencia clara entre ambas variables. Por ello, puede afirmarse que la retroalimentación fue funcional y coherente con la fuerza medida, aunque en nuestra implementación se comportó como una relación por umbral más que como una proporcionalidad lineal estricta con un factor $\beta$.

- A partir de las gráficas del ensayo de contacto se observa que los torques articulares que más cambian durante la interacción son principalmente $\tau_2$ y $\tau_3$, mientras que otras articulaciones permanecen relativamente más estables. Esto es consistente con el hecho de que el esfuerzo principal del manipulador durante la presión del botón recae en las articulaciones responsables del posicionamiento y empuje del brazo. No se realizó un cálculo exacto de $\boldsymbol{\tau}_{esp} = \boldsymbol{J}^\top \hat{\boldsymbol{F}}_{ext}$ en el instante de primer contacto, ya que para ello sería necesario registrar la configuración articular exacta y el Jacobiano numérico en ese momento. Sin embargo, en orden de magnitud sí se observa coherencia entre la presencia de fuerzas elevadas en el contacto y variaciones de torque de varios Nm en las articulaciones más cargadas.

### VII-D. *Análisis de Estabilidad del Sistema Háptico*

*Pregunta E4 [8 pts]:*

(a) Realizaron la prueba de **contacto con pared rígida** (hard wall contact) tanto en simulación (pared simulada con alto $K_d$) como en objeto real. ¿El sistema fue estable en ambos casos? Grafiquen la respuesta de fuerza.

(b) Identifiquen el valor de $\beta_{crit}$ experimental (incrementando $\beta$ hasta observar oscilaciones). ¿Coincide con el valor teórico calculado en C2(a)?

(c) ¿Observaron algún **acoplamiento de inestabilidad** cuando el maestro y el esclavo estaban en contacto simultáneo con sus respectivas superficies? Describan el fenómeno.

- En nuestra implementación no se utilizó una pared virtual clásica basada en una rigidez $K_d$ alta, sino una lógica de contacto por umbral de fuerza. Por esta razón, la prueba de *hard wall contact* en simulación no se realizó en el sentido tradicional de un entorno háptico con impedancia programada. Lo que sí se evaluó fue el contacto con un objeto real a través del sensor de fuerza externo, observando que el sistema permaneció estable en el sentido de que, al superarse el umbral de 6.5 N, el robot maestro entraba en estado de freno y evitaba que el operador siguiera avanzando sobre la superficie. La respuesta de fuerza registrada mostró incrementos claros durante el contacto y una transición definida entre el estado libre y el estado bloqueado, sin crecimiento descontrolado de la señal.

- Tampoco se definió experimentalmente un valor de $\beta_{crit}$, ya que en la versión final del sistema no se implementó una retroalimentación háptica continua del tipo
  $$\boldsymbol{\tau}_{m,fb} = \boldsymbol{J}_m^\top \beta \hat{\boldsymbol{F}}_{ext},$$
  sino una retroalimentación discreta basada en umbral. En consecuencia, no fue posible comparar un $\beta_{crit}$ experimental con el valor teórico planteado en C2(a). En nuestro caso, el parámetro crítico del sistema fue el umbral de activación de fuerza, fijado en 6.5 N, a partir del cual el maestro se bloquea.

- Durante las pruebas no se observó un acoplamiento de inestabilidad fuerte entre maestro y esclavo cuando existía contacto, principalmente porque la acción háptica no era continua sino binaria. El fenómeno que sí apareció fue una sensación de retardo cuando el sistema pasaba del estado libre al estado bloqueado y posteriormente al estado liberado mediante retroceso del operador. Esto significa que, aunque el sistema fue estable para la lógica implementada, una versión futura con realimentación continua requeriría un análisis más cuidadoso de pasividad, retardo de red y amortiguamiento para evitar oscilaciones entre ambos manipuladores.

## VIII. Discusión

*Pregunta D1 [8 pts]:*

(a) **Limitaciones del sistema implementado:** Identifiquen y analicen cuantitativamente al menos **tres limitaciones** de su implementación (e.g., precisión del estimador de fuerza, latencia, espacio de trabajo, etc.). Para cada limitación, propongan una solución concreta y viable con el hardware disponible.

(b) **Comparación con estado del arte:** Comparen sus métricas de desempeño (error de posición, precisión de fuerza, latencia) con al menos DOS trabajos publicados de teleoperación con brazos robóticos similares. ¿En qué aspectos su sistema es competitivo? ¿En cuáles queda rezagado?

(c) **Escalabilidad:** Si tuvieran que operar este sistema con un retardo de $T_d = 200$ ms (e.g., operación transcontinental), ¿qué modificaciones al controlador serían necesarias? Mencionen al menos una técnica del estado del arte (e.g., wave variables, model mediation).

(d) **Aplicación real:** Propongan una aplicación concreta en la industria o medicina donde su sistema, con las mejoras indicadas, podría tener impacto. Estimen el orden de magnitud del costo de implementación.

- Limitaciones del sistema implementado: Se identificaron tres limitaciones principales. La primera es la precisión del estimador de fuerza, ya que la fuerza no se mide directamente en la muñeca del robot, sino que se infiere a partir de la compresión de un resorte usando un sensor VL53L0X y la ley de Hooke. Esto introduce errores por alineación mecánica, resolución del sensor, histéresis del resorte y ruido de medición. Además, la retroalimentación se activa mediante un umbral de 6.5 N, por lo que no es continua sino discreta. Una mejora viable sería recalibrar experimentalmente la constante del resorte, mejorar la guía mecánica y usar un filtrado más robusto. La segunda limitación es la diferencia de frecuencias de actualización: el control del esclavo opera a 200 Hz, mientras que el sensor de fuerza publica a 20 Hz, lo que reduce la resolución temporal de la detección de contacto. Una mejora posible sería aumentar la tasa de publicación del ESP32 y sincronizar mejor el lazo de fuerza con el lazo articular. La tercera limitación es que la retroalimentación háptica es binaria, ya que el maestro se bloquea al superar el umbral, en lugar de reflejar una fuerza continua. Una mejora concreta sería implementar una desaceleración progresiva o una realimentación proporcional acotada.

- Comparación con el estado del arte: En comparación con sistemas de teleoperación bilateral reportados en la literatura, nuestro sistema queda por debajo en transparencia háptica, precisión de fuerza y manejo formal del retardo. Los trabajos del estado del arte suelen emplear retroalimentación continua de fuerza, control bilateral más avanzado y estrategias de compensación de retardo para conservar estabilidad y realismo en la interacción. En contraste, nuestro sistema destaca por su bajo costo, simplicidad de integración y rapidez de implementación, al combinar ROS 2, micro-ROS, ESP32 y un sensor externo. Por lo tanto, puede considerarse competitivo en accesibilidad y facilidad de desarrollo, aunque queda rezagado frente a plataformas más avanzadas en cuanto a precisión, continuidad háptica y desempeño dinámico.

- Escalabilidad: Si el sistema tuviera que operar con un retardo de $T_d = 200$ ms, la estrategia actual sería insuficiente para mantener una interacción háptica transparente. En ese caso sería necesario modificar el controlador para incorporar técnicas de compensación de retardo, como wave variables o model mediated teleoperation, que son enfoques del estado del arte orientados a preservar estabilidad y pasividad bajo retardos grandes. En nuestro sistema esto implicaría sustituir la lógica de bloqueo inmediato por una respuesta más gradual y robusta, además de introducir predicción o mediación de modelo para reducir el efecto del retardo sobre la percepción del operador.

- Aplicación real: Una aplicación realista para este sistema, una vez mejorado, sería la telemanipulación remota en tareas de inspección o contacto delicado en celdas industriales, donde el operador necesite saber cuándo el efector final ha tocado una superficie u objeto. También podría adaptarse a entornos de entrenamiento o demostración en laboratorio. Dado que ya se cuenta con los dos xArm Lite 6, el costo adicional de implementación estaría dominado por sensado, electrónica auxiliar e integración, por lo que el orden de magnitud sería de cientos a pocos miles de dólares, considerablemente menor que el de sistemas hápticos industriales o médicos especializados.

## IX. Conclusiones

*Pregunta Cn1 [6 pts]:* Redacten una sección de conclusiones que responda:

(a) ¿Lograron implementar teleoperación con sensado de fuerza funcional entre los dos xArm Lite 6? ¿Con qué nivel de desempeño (cuantifiquen)?

(b) ¿Cuál fue el mayor desafío técnico encontrado y cómo lo resolvieron?

(c) ¿Qué aprendizaje clave obtuvieron sobre la relación entre el Jacobiano, los torques articulares y la fuerza cartesiana?

(d) ¿Qué harían diferente si repitieran el proyecto?

En este proyecto se logró implementar un sistema funcional de teleoperación con sensado de fuerza entre dos manipuladores xArm Lite 6. El robot esclavo siguió el movimiento del maestro a partir de sus posiciones y velocidades articulares, mientras que la detección de contacto se realizó con un sistema externo basado en un sensor VL53L0X, un resorte y un ESP32 con micro-ROS. En general, el desempeño fue suficiente para demostrar seguimiento articular estable y detección clara de eventos de contacto, especialmente cuando la fuerza superó el umbral

definido para activar el freno del maestro.

El mayor desafío técnico fue obtener retroalimentación háptica sin contar con un sensor comercial de fuerza ni con una interfaz de torque continua en el robot maestro. Para resolverlo, se diseñó una solución alternativa de bajo costo basada en la deformación de un resorte, cuya fuerza se estimó mediante la ley de Hooke y se utilizó para activar una respuesta de bloqueo en el maestro. También fue importante ajustar el controlador PD del esclavo para mantener un seguimiento suave y evitar comandos excesivos.

Como aprendizaje principal, comprendimos mejor la relación entre el movimiento articular, la geometría del manipulador y la fuerza de interacción con el entorno. Si repitiéramos el proyecto, mejoraríamos la validación del sensor de fuerza y buscaríamos implementar una retroalimentación háptica continua, en lugar de una respuesta binaria de bloqueo.

## Agradecimientos

Agradecemos al Computational Robotics Lab por el apoyo brindado para el desarrollo de este proyecto. De igual forma, agradecen al Prof. Alberto Muñoz por la guía académica, las observaciones realizadas durante el curso y el acompañamiento en el proceso de diseño e implementación. Finalmente, se reconoce el trabajo colaborativo del equipo y el uso de las herramientas, recursos de laboratorio y materiales que hicieron posible la realización de este trabajo.

## Referencias

[1] UFACTORY, "xArm-Python-SDK," GitHub repository, 2024. [Online]. Available: https://github.com/xArm-developer/xArm-Python-SDK

[2] B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, Springer, 2009.

[3] M.W. Spong, S. Hutchinson, M. Vidyasagar, *Robot Modeling and Control*, Wiley, 2006.

[4] N. Hogan, "Impedance Control: An Approach to Manipulation," *ASME J. Dyn. Sys., Meas., Control*, vol. 107, 1985.

[5] J.E. Colgate, J.M. Brown, "Factors affecting the Z-width of a haptic display," in *Proc. IEEE ICRA*, 1994, pp. 3205–3210.

[6] R.M. Murray, Z. Li, S.S. Sastry, *A Mathematical Introduction to Robotic Manipulation*, CRC Press, 1994.

[7] G. Niemeyer, C. Preusche, G. Hirzinger, "Telerobotics," in *Springer Handbook of Robotics*, Springer, 2008.

[8] S. Thompson, I. Reid, **A. Muñoz**, D.W. Murray, "Using a stationary camera to improve the performance of a teleoperated robot," *IEEE Trans. Systems, Man & Cybernetics-A*, vol. 31(1), pp. 74–82, 2001.

[9] A. Mueller, "Modern Robotics: Mechanics, Planning, and Control [Bookshelf]," *IEEE Control Systems Magazine*, vol. 39, no. 6, pp. 100–102, Dec. 2019.

[10] Y. Hao, H. Zhang, Z. Zhang, C. Hu, C. Shi, "Development of Force Sensing Techniques for Robot-Assisted Laparoscopic Surgery: A Review," *IEEE Transactions on Medical Robotics and Bionics*, vol. 6, no. 3, pp. 868–887, Aug. 2024.

[11] K. Darvish *et al.*, "Teleoperation of Humanoid Robots: A Survey," *IEEE Transactions on Robotics*, vol. 39, no. 3, pp. 1706–1727, Jun. 2023.

[12] Oscar.E. Ramos, "Cinemática Diferencial de Robots Manipuladores," notas de clase, Curso *Fundamentos de Robótica*, Universidad de Ingeniería y Tecnología (UTEC), 2021. [Online]. Available: https://oramosp.epizy.com/teaching/211/fund-robotica/clases/6_Cinematica_Diferencial.pdf

## Vínculos Importantes

Repositorio en Github: https://github.com/Jose05M/reto_Teleoperacion_Haptica

Carpeta de los videos de los experimentos: https://drive.google.com/drive/folders/1Bs8ZEHBYy82A9pPPE2zRgDqqtY2mnA_w?usp=drive_link